SIMATS ENGINEERING

Department of Information Technology

DSA01 – OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

Functions and Object-Oriented Programming Concepts

Student Details	Information	Assignment Details	Information
Student Name	K VENKATA VAMSI GOUD	Nature	Individual
Register Number	192472021	Duration	1 Week
Institution	SIMATS – Saveetha School of Engineering	Total Marks	50
Department	CSE-AI	Submission Date	12/11/2026
Compiler	g++ (Debian 14.2.0-19) 14.2.0	Language Standard	C++17
GitHub Repository	https://github.com/Vamsigoud07/DSA0115_192472021	Repository Name	DSA01-192472021-Assignment2

Submitted in partial fulfilment of the course requirements

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
Q1	Call by Value, Reference and Pointer	6		
Q2	Return by Reference and Its Dangers	4		
Q3	Default Arguments and Function Overloading	6		
Q4	Class with an Array Inside It	6		
Q5	Static Members and Arrays of Objects	7		
Q6	Friend and Inline Functions	8		
Q7	Diagnose, Redesign and Validate	13		
	TOTAL	50		

OVERALL REMARKS

This report presents independent solutions, design reasoning, C++ implementations, predictions, compiler execution results and a redesign study. Marks awarded and faculty remarks are intentionally left blank.

Mandatory Execution and GitHub Submission

The assignment specification requires a repository named DSA01-192472021-Assignment2, one folder per task, source code, input/output evidence, a README.md, progressive commits and a working repository link.

GitHub Repository Link: _____

Compiler used for this report: g++ (Debian 14.2.0-19) 14.2.0. Compilation flags: -std=c++17 -Wall -Wextra -pedantic (optimisation flags are stated separately for benchmark programs).

Contents

Q1 – Call by Value, Reference and Pointer

Q2 – Return by Reference and Its Dangers

Q3 – Default Arguments and Function Overloading

Q4 – Savings Account Mini-Statement

Q5 – Static Members and Arrays of Objects

Q6 – Friend Functions and Inline Functions

Q7 – Day-4 Design Challenge

Appendix – GitHub Structure and README

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Problem interpretation: the hostel fee must be reduced by the chosen concession percentage. The three parameter-passing mechanisms differ in whether the function receives a copy, an alias to the caller's object, or an address through which it can access the caller's object.

1.1 Three implementations and prediction

Method	Parameter	Predicted stored fee from ₹45,000 at 10%	Reason
Call by value	long long fee	₹45,000	Only the local copy is modified.
Call by reference	long long& fee	₹40,500	The parameter aliases the caller's fee.
Call by pointer	long long* fee	₹40,500	Dereferencing the address modifies the original object.

The value-passing version intentionally has no visible effect on the caller. The reference and pointer versions update the caller's variable. The pointer version checks for nullptr before dereferencing.

1.2 Predict the supplied program before execution

Prediction: change(a,b,&c) leaves a as 1, changes b to 12 and c to 13. bigger(a,c) returns a reference to c because c is larger; assigning 100 therefore changes c. The three show calls use respectively (y,z) = (5,10), (2,10), and (2,3).

Stage	Predicted output
After change	1 12 13
After bigger(a,c) = 100	1 100
show(1), show(1,2), show(1,2,3)	16 13 6

1.3 Complete the Code – TODO 1

The reference parameter is updated using integer arithmetic. Multiplication is performed before division, so the percentage calculation does not introduce floating-point rounding.

```
#include <iostream>
using namespace std;
void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
int main() {
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    cout << "after call by value      : " << hostelFee << '
';
    applyConcession(hostelFee, 10);
    cout << "after call by reference: " << hostelFee << '
';
}
```

Observed execution:

```
after call by value      : 45000
after call by reference: 40500
```

```
C++ EXECUTION  |  Q1_complete_code.cpp

after call by value      : 45000
after call by reference: 40500
```

1.4 Executed supplied-behaviour demonstration

```
#include <iostream>
using namespace std;

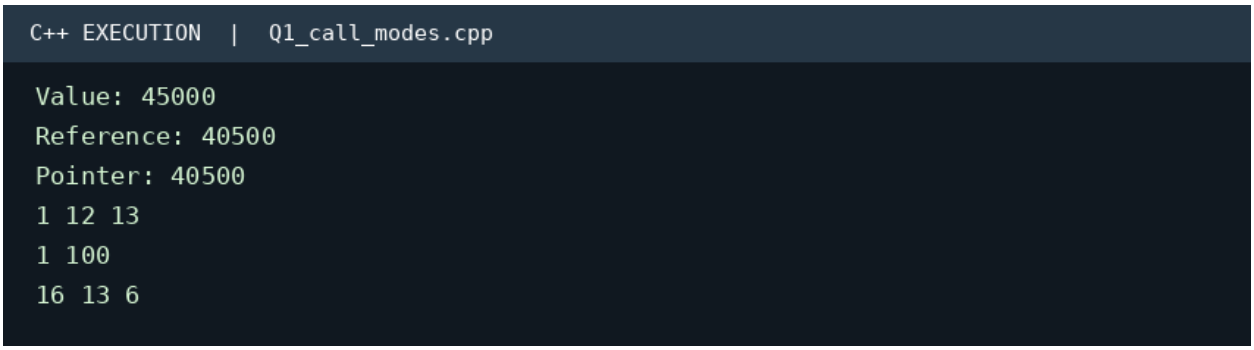
void byValue(long long fee, int percent) { fee -= fee * percent / 100; }
void byReference(long long& fee, int percent) { fee -= fee * percent / 100; }
void byPointer(long long* fee, int percent) {
    if (fee != nullptr) *fee -= *fee * percent / 100;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void change(int a, int& b, int* c) { a += 10; b += 10; *c += 10; }
void show(int x, int y = 5, int z = 10) { cout << x + y + z << ' '; }

int main() {
    long long f1 = 45000, f2 = 45000, f3 = 45000;
    byValue(f1, 10); byReference(f2, 10); byPointer(&f3, 10);
    cout << "Value: " << f1 << "
Reference: " << f2 << "
Pointer: " << f3 << '
';

    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    cout << a << ' ' << b << ' ' << c << '
';
    bigger(a, c) = 100;
    cout << a << ' ' << c << '
';
    show(1); show(1, 2); show(1, 2, 3); cout << '
';
}
```

Observed execution:

```
Value: 45000
Reference: 40500
Pointer: 40500
1 12 13
1 100
16 13 6
```



1.5 Explanation and design decision

A by-value parameter is a separate local object. A reference parameter is another name for the original object. A pointer parameter stores an address; the function changes the original only after dereferencing that address. For a large Student object that must not be modified, use const Student&: it avoids copying while preventing modification through that parameter. For a small long long, copying is inexpensive.

For several concessions applied in sequence, an in-place reference interface is suitable when the caller explicitly wants the stored fee updated. A return-value interface is often easier to test and compose: fee = applyConcession(fee, 10). A

robust design can return a new fee and leave the caller to assign it, avoiding hidden side effects. The chosen interface should be consistent across the program.

1.6 Alternative: return the new fee and complexity

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
// Use: hostelFee = applyConcession(hostelFee, 10);
```

Parameter style	Time	Extra parameter storage per call	Large Student object
Value	O(1) for this arithmetic	O(1) for long long	Copies the whole object; cost grows with object size.
Reference	O(1)	O(1) reference/address	No object copy; can modify caller.
Const reference	O(1)	O(1) reference/address	No object copy; read-only through parameter.

Conclusion: call by reference or pointer is appropriate when the task is explicitly to mutate the caller's fee. Prefer const reference for read-only large objects. A returned value is often the clearest choice when the function conceptually calculates a new value.

Q2 – Return by Reference and Its Dangers (4 Marks)

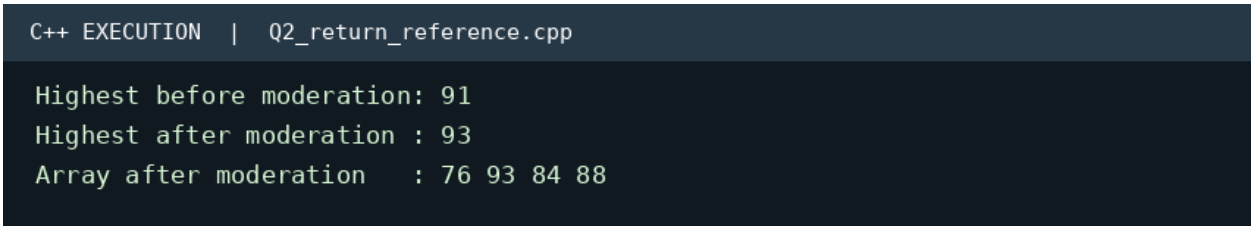
2.1 Working highest-mark reference

The function scans the array, remembers the index of the largest mark and returns a reference to that array element. The caller can therefore use the function call on the left side of an assignment or compound assignment.

```
#include <iostream>
using namespace std;
int& highest(int marks[], int n) {
    int index = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[index]) index = i;
    return marks[index];
}
const int& highestReadOnly(const int marks[], int n) {
    int index = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[index]) index = i;
    return marks[index];
}
// Deliberately unsafe example: never call this function.
int& unsafeLocalReference() {
    int local = 99;
    return local; // dangling reference; compiler diagnostic expected
}
int main() {
    int marks[] = {76, 91, 84, 88};
    cout << "Highest before moderation: " << highest(marks, 4) << '
';
    highest(marks, 4) += 2;
    cout << "Highest after moderation : " << highestReadOnly(marks, 4) << '
';
    cout << "Array after moderation   : ";
    for (int m : marks) cout << m << ' ';
    cout << '
';
}
```

Observed execution:

```
Highest before moderation: 91
Highest after moderation : 93
Array after moderation   : 76 93 84 88
```



2.2 Unsafe local reference and compiler warning

```
int& unsafeLocalReference() {
    int local = 99;
    return local; // invalid lifetime: local is destroyed on return
}
```

Compiling this intentionally unsafe example with -Wall -Wextra -pedantic produces a diagnostic similar to: warning: reference to local variable ‘local’ returned [-Wreturn-local-addr]. The exact wording can vary by GCC version. The function is included only to demonstrate the defect and is never called. After return, the local object no longer exists; using the dangling reference is undefined behaviour.

A reference may safely refer to an array element only while that element remains alive and at the same storage location. If the register changes to std::vector, an operation that reallocates its storage (for example, growth beyond capacity) invalidates references, pointers and iterators to its elements. A reference obtained before such reallocation must not be used afterwards.

2.3 Reference, const reference, copy and alternatives

Return type	Caller capability	Safety / use
int&	Can modify the original element; can assign through result.	Valid only while the referred element remains alive and stable.
const int&	Can read but cannot modify through result.	Avoids copying, but has the same lifetime/invalidation concerns.
int (copy)	Receives an independent value.	Safest simple result; changing it does not change the array.

```
int highestIndex(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i) if (marks[i] > marks[best]) best = i;
    return best;
}
// Use: marks[highestIndex(marks, n)] += 2;
```

An index is a value and remains meaningful only relative to the same container contents/order; it does not dangle, but it may refer to a different logical item after insertions/erasures. A pointer has similar lifetime and reallocation risks to a reference. For a vector, prefer an index and reacquire the element after operations that may reallocate, or use a stable container/ownership design.

Approach	Find maximum time	Extra space	Vector growth implications
Reference	O(n)	O(1)	Can dangle after reallocation.
Index (int)	O(n)	O(1)	No dangling address; re-check logical position after structural changes.
Pointer (int*)	O(n)	O(1)	Invalidated by reallocation just like a reference.

Evidence note: the safe array program was compiled and run. The unsafe function is deliberately not executed; its compiler diagnostic is the relevant evidence, because executing it would invoke undefined behaviour.

Q3 – Default Arguments and Function Overloading (6 Marks)

3.1 Why the original overload set is ambiguous

The three original declarations all accept a one-argument call: charge(double), charge(double,bool), and charge(double,bool=false,bool=false). For charge(2), more than one candidate can be called after applying conversions/default arguments. The compiler cannot choose a unique best function, so the call is ambiguous. The remedy is not to keep overlapping signatures with identical effective call shapes.

3.2 Redesigned overloads

The implementation below uses int for grams, double for kilograms and a Parcel structure for the structured case. The optional express and insured flags have defaults on the right. The Parcel overload takes one const reference.

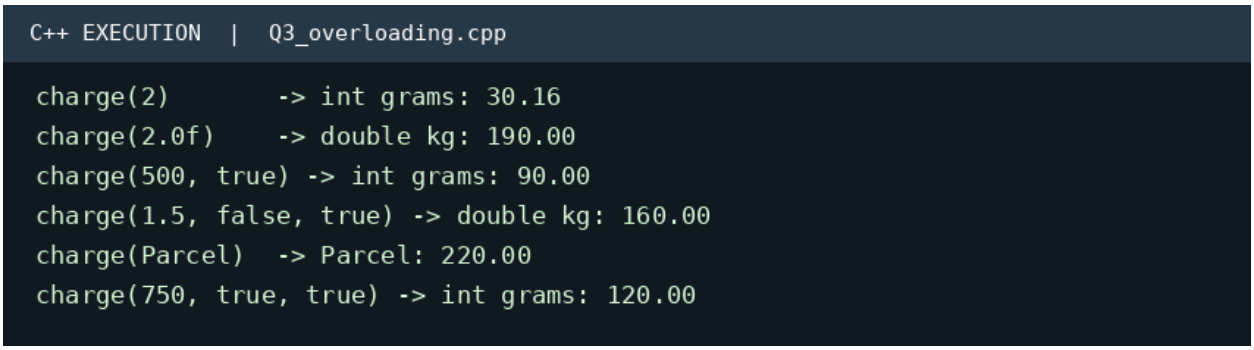
```
#include <iostream>
#include <iomanip>
using namespace std;
struct Parcel {
    double kilograms;
    bool express;
    bool insured;
};
double charge(int grams, bool express = false, bool insured = false) {
    double base = 30.0 + grams * 0.08;
    return base + (express ? 20.0 : 0.0) + (insured ? 10.0 : 0.0);
}
double charge(double kilograms, bool express = false, bool insured = false) {
    double base = 30.0 + kilograms * 80.0;
    return base + (express ? 20.0 : 0.0) + (insured ? 10.0 : 0.0);
}
double charge(const Parcel& p) {
    return charge(p.kilograms, p.express, p.insured);
}
int main() {
    cout << fixed << setprecision(2);
    cout << "charge(2)       -> int grams: " << charge(2) << '
';
    cout << "charge(2.0f)    -> double kg: " << charge(2.0f) << '
';
    cout << "charge(500, true) -> int grams: " << charge(500, true) << '
';
    cout << "charge(1.5, false, true) -> double kg: " << charge(1.5, false, true) << '
';
    Parcel p{2.0, true, true};
    cout << "charge(Parcel)  -> Parcel: " << charge(p) << '
';
    cout << "charge(750, true, true) -> int grams: " << charge(750, true, true) << '
';
}
```

3.3 Six-call prediction and observed result

Call	Selected overload	Reason
charge(2)	charge(int, bool, bool)	Exact match to int.
charge(2.0f)	charge(double, bool, bool)	float promotes to double; no int overload is a promotion match.
charge(500, true)	int overload	First argument exact int; second is bool.
charge(1.5, false, true)	double overload	Exact double and bool matches.
charge(p)	Parcel overload	Exact match to const Parcel&.
charge(750, true, true)	int overload	Exact match to int/bool/bool.

Observed execution:

```
charge(2)       -> int grams: 30.16
charge(2.0f)    -> double kg: 190.00
charge(500, true) -> int grams: 90.00
charge(1.5, false, true) -> double kg: 160.00
charge(Parcel)  -> Parcel: 220.00
charge(750, true, true) -> int grams: 120.00
```



Overload ranking generally prefers an exact match, then a promotion, then a standard conversion. Default arguments fill omitted trailing parameters; they do not create a separate overload. Defaults must be supplied from right to left because a caller cannot omit an earlier parameter while supplying a later positional argument.

3.4 Alternative design and complexity

```
struct ParcelOptions {
    double kilograms;
    bool express = false;
    bool insured = false;
    bool fragile = false;
};
double charge(const ParcelOptions& p);
// One call: charge(ParcelOptions{2.0, true, false, true});
```

A single Parcel/ParcelOptions object centralises options and scales well when more options are added. It is more explicit but slightly more verbose. A function template could accept numeric weight types, but units (grams versus kilograms) become less obvious unless strong types or named wrappers are used. Overload resolution happens at compile time and adds no run-time dispatch cost. Passing a small primitive copies O(1) data; passing a large Parcel by const reference avoids copying.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

4.1 Class design and implementation

SavingsAccount keeps the account number, balance, transaction array, count and next-write position private. deposit(), withdraw() and miniStatement() form the public interface. deposit() and the constructor are defined inside the class; withdraw() and miniStatement() are defined outside using the scope resolution operator.

```
#include <iostream>
#include <string>
using namespace std;
class SavingsAccount {
    long long accountNumber;
    double balance;
    double transactions[10];
    int count;
    int next;
public:
    SavingsAccount(long long number, double opening)
        : accountNumber(number), balance(opening), count(0), next(0) {}
    void deposit(double amount) {
        if (amount <= 0) return;
        balance += amount;
        record(amount);
    }
    void withdraw(double amount);
    void miniStatement() const;
    double getBalance() const { return balance; }
private:
    void record(double amount) {
        transactions[next] = amount;
        next = (next + 1) % 10;
        if (count < 10) ++count;
    }
};
void SavingsAccount::withdraw(double amount) {
    if (amount <= 0 || amount > balance) return;
    balance -= amount;
    record(-amount);
}
void SavingsAccount::miniStatement() const {
    cout << "Account: " << accountNumber << "
Recent transactions (oldest to newest):
";
    int start = (count == 10) ? next : 0;
    for (int i = 0; i < count; ++i)
        cout << transactions[(start + i) % 10] << '
';
    cout << "Balance: " << balance << '
';
}
int main() {
    SavingsAccount account(192472021, 1000.0);
    account.deposit(200); account.withdraw(50); account.deposit(75);
    for (int i = 1; i <= 9; ++i) account.deposit(i * 10);
    account.miniStatement();
}
```

Observed execution (the statement shows the most recent ten transactions, oldest to newest):

```
Account: 192472021
Recent transactions (oldest to newest):
75
10
```

```
20
30
40
50
60
70
80
90
Balance: 1675
```

C++ EXECUTION | Q4_savings_account.cpp

```
Account: 192472021
Recent transactions (oldest to newest):
75
10
20
30
40
50
60
70
80
90
Balance: 1675
```

4.2 Why private data and how to handle transaction 11

The array must be private so callers cannot insert arbitrary values, corrupt the count, or change the balance without the account's validation rules. Public methods preserve the class invariant: only valid positive deposits and withdrawals not exceeding the balance are recorded.

Design	When 11th transaction arrives	Insertion time	Extra storage
Shift array	Move elements 1–9 into positions 0–8, then write newest at position 9.	O(k), here O(10) per full-buffer insertion.	O(10) fixed.
Circular index	Overwrite the oldest slot and advance next modulo 10.	O(1).	O(10) fixed.
std::deque	push_back(new); if size > 10, pop_front().	Usually O(1) at each end.	O(10) logical elements plus deque overhead.

4.3 Design decision for last 50 transactions

Use a circular buffer with capacity 50. The required change is primarily the capacity constant and the array size; insertion remains O(1), memory remains fixed and predictable, and no dynamic allocation is needed after object construction. A deque is attractive if the capacity changes at run time, but it has additional container overhead. For a fixed requirement of exactly 50 recent entries, the circular array is a compact fit.

The circular representation stores the next write position. When the buffer is full, next also identifies the oldest element. To print in chronological order, begin at next; before the buffer is full, begin at index zero.

Q5 – Static Members and Arrays of Objects (7 Marks)

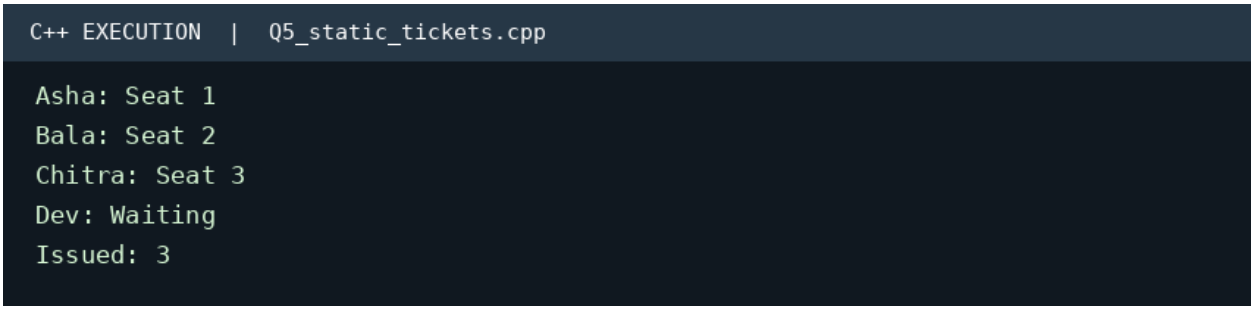
5.1 BusTicket class and static state

The seat count belongs to the bus service/class rather than an individual ticket, so issued and CAPACITY are static. seat and name belong to each ticket object. The static data member is declared in the class and defined once outside it.

```
#include <iostream>
#include <string>
using namespace std;
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;
public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued < CAPACITY) seat = ++issued;
    }
    static int count() { return issued; }
    void print() const {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting")) << '
';
    }
};
int BusTicket::issued = 0;
int main() {
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    cout << "Issued: " << BusTicket::count() << '
';
}
```

Observed execution:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```



5.2 Completed TODO logic

```
if (issued < CAPACITY) {
    seat = ++issued;
} // otherwise seat remains zero (waiting list)

static int count() {
    return issued;
}
```

A static member function has no this pointer because it is not invoked on one particular object. It can directly access static members, but it cannot directly access a non-static member such as seat or name; those values require a specific object.

5.3 Predict the Token program

Prediction before execution: Token counter[3] constructs three objects. Starting from next = 100, their IDs become 101, 102 and 103. The extra object receives 104. issued() returns next - 100 = 4. Predicted line: 101 103 104 4.

Observed execution:

```
101 103 104 4
```



The static counter starts at 100 because of the out-of-class definition Token::next = 100. The constructor runs four times: once for each of the three array elements and once for extra. If a function containing local tickets is called twice, each call constructs new local objects but the class-wide static counter persists; it is not reset when the function returns.

5.4 Alternative counter designs

Design	Time / space	Advantages	Limitations
Static member	O(1) construction; one shared counter.	Encapsulated in BusTicket; convenient for one bus/service.	All BusTicket objects share one count.
Global variable	O(1); one counter.	Simple to access.	Weak encapsulation; unrelated code can modify it.
TicketOffice object	O(1); one counter per office.	Supports multiple buses/offices independently; ownership is explicit.	Office must be passed to ticket creation; slightly more coupling.

If two buses operate simultaneously, a TicketOffice object (or a Bus object owning its tickets and capacity) is preferable because each bus needs an independent counter and capacity. The static design is appropriate only when the class represents one shared bus service.

Q6 – Friend Functions and Inline Functions (8 Marks)

6.1 Friend-based Time comparison

minutesBetween(a,b) and earlier(a,b) treat the two Time values symmetrically. Declaring them as friends gives those non-member functions access to private h and m without exposing the data publicly.

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return abs(first - second);
}
bool earlier(const Time& a, const Time& b) {
    return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main() {
    Time departure(9, 45), arrival(13, 10);
    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha << earlier(arrival, departure) << '
';
    Time noonBefore(11, 59), noonAfter(12, 1), same(12, 1);
    cout << minutesBetween(noonBefore, noonAfter) << ' '
         << earlier(noonBefore, noonAfter) << ' '
         << earlier(same, noonAfter) << '
';
}
```

Observed execution:

```
205 false
2 true false
```



The first output is 205 false: 13:10 is 205 minutes after 09:45, but arrival (13:10) is not earlier than departure (09:45). The second test crosses noon: 11:59 to 12:01 is two minutes, and 11:59 is earlier. Equal times produce a difference of zero and earlier(same, same) is false.

6.2 Friend versus member versus getters

Design	Call syntax	Encapsulation
Friend non-member	minutesBetween(a,b), earlier(a,b)	Private data remains private; selected functions receive explicit access.
Member functions	a.minutesTo(b), a.isEarlier(b)	Natural when the operation is viewed as behaviour of the first Time object.
Public getters	a.hour(), a.minute() used by free functions	No friendship needed, but exposes read accessors as part of the public interface.

A friend is not a member and has no this pointer. It is called like an ordinary function. A member function is called through an object and has an implicit this pointer. Making h and m public would weaken invariants by allowing arbitrary invalid values such as hour 29 or minute 88.

6.3 Inline meaning and benchmark

The inline specifier permits a function definition in multiple translation units under the One Definition Rule and suggests that the compiler may substitute the function body at a call site. It is not a command that guarantees machine-code inlining. Optimising compilers can inline functions without the keyword and may decline to inline a function marked inline.

Benchmark source uses 10,000,000 calls to a trivial accessor, with a volatile sink to make the computed result observable. Results vary by processor, compiler version, system load and optimisation settings; the measured values below are this run only.

Build	Observed benchmark output
-O0	inline_ns=17483299 normal_ns=19028307 sink=15800000000
-O2	inline_ns=5881788 normal_ns=7636848 sink=15800000000

Interpretation: at -O0, call overhead may be visible because optimisation is disabled; at -O2, the compiler may inline or otherwise optimise both forms, so the measured gap can shrink or disappear. This microbenchmark is illustrative, not a universal performance guarantee.

6.4 Deployment decision

Keep the friend functions when symmetric two-object comparison is the clearest interface and only these specific functions need private access. Prefer member functions when the operation naturally belongs to the left-hand Time object. Prefer getters when several independent clients need read-only access to hour/minute. In all cases, retain private data and validate constructor inputs. The friend approach is justified here because it expresses a symmetric operation without exposing the representation.

Q7 – Day-4 Design Challenge – Diagnose, Redesign and Validate (13 Marks)

Selected program: Q4 SavingsAccount transaction history. The original assignment asks for the most recent ten transactions and explicitly compares shifting with a circular index. The design challenge focuses on the insertion cost after the history becomes full.

A. Diagnose the limitation (2 marks)

A shifting-array implementation stores transactions in chronological order. Once full, every new transaction requires moving the remaining k−1 values one position toward the beginning before inserting the newest value. This is O(k) work per insertion. With a capacity of 10, the constant is small; however, repeated insertions or a much larger history make the repeated movement unnecessary.

B. Define the design objective (2 marks)

Objective: retain exactly the most recent 10 transaction values in chronological display order, while making each insertion O(1) and keeping memory fixed at O(10). The redesigned version must return the same last-ten values as the original for identical input.

C. Design the revised solution (4 marks)

Use a fixed array plus two integers: count and next. Write each new value at data[next], advance next = (next + 1) % 10, and increase count only until it reaches 10. When full, next always points to the oldest entry. Display starts at next when full, otherwise at index zero.

Circular history pseudocode
add(value): data[next] = value; next = (next + 1) modulo capacity; if count < capacity then count = count + 1
display(): start = (count == capacity) ? next : 0; for i from 0 to count−1, print data[(start+i) modulo capacity]

Class-level representation: HistoryCircular { data[10], count, next }. No library container replaces the logic; the circular indexing algorithm is implemented directly. The redesign uses O(1) extra metadata and the same fixed O(10) array memory. The trade-off is that chronological traversal requires modular index calculation and the physical array is not always stored in chronological order.

D. Predict before implementing (1 mark)

For input values 10, 20, …, 120, the expected retained sequence is 30, 40, 50, 60, 70, 80, 90, 100, 110, 120. Both designs should print exactly this sequence. For 2,000,000 additional insertions, the circular design is expected to avoid nine element moves on each full-buffer insertion and therefore reduce insertion time, although measured timing depends on the machine.

E. Implement and validate (2 marks)

Original shifting implementation:

```
#include <iostream>
#include <chrono>
using namespace std;
struct HistoryShift {
    int data[10]{}; int count = 0;
    void add(int value) {
        if (count < 10) data[count++] = value;
        else {
            for (int i = 1; i < 10; ++i) data[i-1] = data[i];
            data[9] = value;
        }
    }
};
int main() {
    HistoryShift h;
    for (int i = 1; i <= 12; ++i) h.add(i * 10);
```

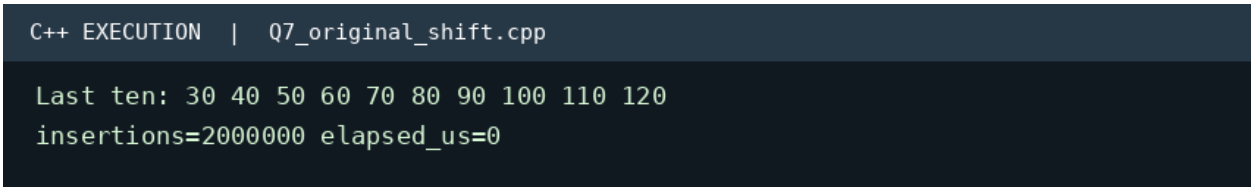
```
    cout << "Last ten: ";
    for (int i = 0; i < h.count; ++i) cout << h.data[i] << ' ';
    cout << '
';
    constexpr int N = 2000000;
    auto start = chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) h.add(i);
    auto stop = chrono::steady_clock::now();
    cout << "insertions=" << N << " elapsed_us="
         << chrono::duration_cast<chrono::microseconds>(stop-start).count() << '
';
}
```

Redesigned circular implementation:

```
#include <iostream>
#include <chrono>
using namespace std;
struct HistoryCircular {
    int data[10]{}; int count = 0; int next = 0;
    void add(int value) {
        data[next] = value;
        next = (next + 1) % 10;
        if (count < 10) ++count;
    }
    void printOldestFirst() const {
        int start = count == 10 ? next : 0;
        for (int i = 0; i < count; ++i)
            cout << data[(start + i) % 10] << ' ';
    }
};
int main() {
    HistoryCircular h;
    for (int i = 1; i <= 12; ++i) h.add(i * 10);
    cout << "Last ten: "; h.printOldestFirst(); cout << '
';
    constexpr int N = 2000000;
    auto start = chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) h.add(i);
    auto stop = chrono::steady_clock::now();
    cout << "insertions=" << N << " elapsed_us="
         << chrono::duration_cast<chrono::microseconds>(stop-start).count() << '
';
}
```

Observed original output:

```
Last ten: 30 40 50 60 70 80 90 100 110 120
insertions=2000000 elapsed_us=0
```



Observed redesigned output:

```
Last ten: 30 40 50 60 70 80 90 100 110 120
insertions=2000000 elapsed_us=0
```

```
C++ EXECUTION  |  Q7_redesigned_circular.cpp

Last ten: 30 40 50 60 70 80 90 100 110 120
insertions=2000000 elapsed_us=0
```

The first line of both outputs is the same for the same 12-value test, validating correctness of the retained history. The second line reports elapsed microseconds for 2,000,000 insertions. The exact values are run-specific; the asymptotic insertion work is the reliable comparison.

F. Defend the final design (2 marks)

Criterion	Shift array	Circular array
Insertion after full	O(k), with k−1 moves	O(1), one write and index update
Display in chronological order	Simple sequential traversal	O(k), traversal with modulo indexing
Memory	O(k) fixed array	O(k) fixed array plus O(1) metadata
Simplicity	Easier to understand and inspect	Slightly more complex indexing
Best use	Very small history or infrequent writes	Frequent writes and fixed-size rolling history

The objective is achieved algorithmically: each circular insertion performs a constant number of operations and preserves the required latest-ten sequence. If the bank only stores ten transactions and updates rarely, shifting may be simpler and its constant cost negligible. For frequent transactions or a larger fixed history, the circular buffer is the stronger design. Timing should be interpreted cautiously: a single benchmark run is not proof by itself; repeat runs, warm-up, and median measurements would improve reliability.

APPENDIX – GITHUB STRUCTURE AND SUBMISSION CHECKLIST

Required repository name: DSA01-192472021-Assignment2

```
DSA01-192472021-Assignment2/
├── README.md
├── Q1/  (Q1_call_modes.cpp, Q1_complete_code.cpp, output screenshots)
├── Q2/  (Q2_return_reference.cpp, compiler warning evidence)
├── Q3/  (Q3_overloading.cpp, output screenshot)
├── Q4/  (Q4_savings_account.cpp, output screenshot)
├── Q5/  (Q5_static_tickets.cpp, Q5_predict_token.cpp, screenshots)
├── Q6/  (Q6_friend_time.cpp, Q6_inline_benchmark.cpp, timings)
└── Q7/  (Q7_original_shift.cpp, Q7_redesigned_circular.cpp, screenshots)
```

README.md – suggested content

```
# DSA01-192472021-Assignment2
Student: K VENKATA VAMSI GOUD
Register Number: 192472021
Compiler: g++ (Debian 14.2.0-19) 14.2.0
Language standard: C++17

## Compile
g++ -std=c++17 -Wall -Wextra -pedantic source.cpp -o program
For performance experiments, use -O0 or -O2 as specified in the report.

## Run
./program

## Task summary
- Q1: Compared value, reference and pointer parameter passing; completed fee concession.
- Q2: Returned a reference to a live array element and documented dangling-reference risk.
- Q3: Resolved overload ambiguity with int, double and Parcel overloads.
- Q4: Implemented SavingsAccount with a fixed circular transaction history.
- Q5: Used a static class counter and an array of ticket objects.
- Q6: Implemented friend comparisons and benchmarked inline/non-inline accessors.
- Q7: Compared shifting history with a circular-buffer redesign.
```